ARGROUTER: Routing LLM Queries on Expected Utility with Real Billed Cost and Reasoning Effort

Hamza Harun Ercul
harunercul5@gmail.com

Abstract

LLM routers choose, for each query, which model should answer. Most compare models by their list price per token, but the bill for a query is dominated by output tokens, including hidden reasoning tokens, and output length differs by more than an order of magnitude between models on the same query. We present ARGROUTER, a router that treats each candidate as a (model, reasoning effort) pair and selects $\arg\max_m \hat{p}_m(x) - \lambda\, \hat{c}_m(x)$, where $\hat{p}_m$ is the predicted probability of a correct answer and $\hat{c}_m$ is the cost that model actually billed on similar queries. Both are estimated from the question content alone, using a small local embedding model, a per-model logistic regression and a nearest-neighbour vote, all fitted on 3,562 held-out calibration items from public benchmarks. The candidate pool and λ are chosen by greedy forward selection and cross-validation on the calibration data. On RouterArena, an open leaderboard of 8,400 queries from 23 benchmarks, ARGROUTER reaches an arena score of 76.30 (accuracy 79.36%, \$0.546 per 1,000 queries) in the benchmark's automated evaluation, against 76.28 for the highest-ranked published router at the time of writing. We report the measurements behind the design: list price mis-ranks models once reasoning tokens are counted; lowering reasoning effort halves cost at unchanged accuracy for one strong model; frontier models do not improve accuracy on this query distribution; and routing on question content alone loses nothing compared with routing on the full prompt.

1 Introduction

A deployment that can call many large language models faces the same decision on every request: which model should answer, given that the strongest model is rarely needed and always the most expensive. LLM routers make this decision per query [2, 4, 6, 11]. Their quality is usually summarised by an accuracy–cost trade-off, and RouterArena [9] now provides an open, standardised leaderboard for it.

Routers need an estimate of what each candidate will cost on the current query. The common approach ranks models by their list price, typically the sum of the input and output price per token. Two facts make that estimate unreliable. First, output tokens dominate the bill for most workloads, and the number of output tokens a model produces is a property of the model, not of the price list: on our calibration data the median output length ranges from 67 to 271 tokens across models, and the 90th percentile from 169 to over 4,300 (Table 3). Second, current models spend part of their output on reasoning tokens that are billed but not shown, and several APIs let the caller set how much reasoning to spend [13]. The same model at a lower reasoning effort is, for routing purposes, a different and cheaper candidate.

ARGROUTER builds both facts into the routing decision:

- **Cost from billed usage.** For each candidate, the expected cost of a query is the cost that

candidate actually incurred on the most similar calibration queries, reasoning tokens included, priced at the evaluation's price table.

- **Reasoning effort as a choice.** A candidate is a (model, effort) pair, for example gemini-3.8-flash at low effort or gemini-3-flash with reasoning switched off.
- **Expected-utility decision.** The router picks $\arg\max_m \hat{p}_m(x) - \lambda\, \hat{c}_m(x)$ and chooses λ and the candidate pool by cross-validation on held-out calibration data.
- **Content-only signal.** The router reads only the question content. It removes instruction and answer-format text with generic rules and never reads benchmark configuration or template files.

On RouterArena's 8,400 queries ARGROUTER scores 76.30 in the benchmark's automated evaluation (Section 6). Beyond the score, our contribution is a set of measurements that explain where the gain comes from and where it does not (Section 7).

2 Related work

Learned routers. RouteLLM [11] trains routers between a strong and a weak model from preference data. Hybrid LLM [4] routes on predicted quality gaps. RouterDC [3] learns query and model embeddings with contrastive objectives, GraphRouter [5] models queries, tasks and models as a graph, and CARROT [14] estimates both accuracy and cost per model and chooses the best trade-off, which is the closest in spirit to our decision rule. Semantic routers such as vLLM Semantic Router [15] route on task signals. Commercial routers include Not Diamond [10], which powers OpenRouter's automatic routing [12].

Cascades and ensembles. FrugalGPT [2] and AutoMix [1] call models in sequence and stop when an answer is judged reliable. ThriftLLM [7] selects ensembles under a budget for classification queries. ARGROUTER makes one call per query; RouterArena scores single-call routing.

Benchmarks. RouterBench [6] released a response matrix for offline router evaluation. RouterArena [9] evaluates routers on a fixed query set with live inference, real prices and a single score that weights accuracy and log-scaled cost, and its maintainers enforce an evaluation-only rule (Section 8).

3 Problem setting

A router receives a prompt x and returns one candidate $m \in \mathcal{M}$. The candidate answers x, its answer is scored $s_m(x) \in [0, 1]$ and it bills $n_m^{\text{in}}(x)$ input and $n_m^{\text{out}}(x)$ output tokens (output includes reasoning). With table prices $\pi_m^{\text{in}}, \pi_m^{\text{out}}$, the cost of the call is

$$c_m(x) = n_m^{\text{in}}(x)\, \pi_m^{\text{in}} + n_m^{\text{out}}(x)\, \pi_m^{\text{out}}. \tag{1}$$

RouterArena aggregates accuracy A and the average cost per 1,000 queries $\bar{c}$ into

$$S = \frac{(1+\beta)\, A\, C}{\beta A + C}, \qquad C = \frac{\log_2 c_{\max} - \log_2 \bar{c}}{\log_2 c_{\max} - \log_2 c_{\min}}, \tag{2}$$

with $\beta = 0.1$, $c_{\max} = 200$ and $c_{\min} = 0.0044$ (USD per 1,000 queries). Because β is small, accuracy dominates; halving cost is worth roughly one accuracy point near our operating point.

S is not additive over queries, so no per-query rule maximises it exactly. We use the linearised rule

$$m^\star(x) = \arg\max_{m \in \mathcal{M}} \; \hat{p}_m(x) - \lambda \, \hat{c}_m(x), \tag{3}$$

and choose λ by maximising S in cross-validation.

4 Method

Question content. The router sees the prompt as the benchmark sends it. It splits the prompt into paragraphs, drops the first and the last (in practice the instruction and the answer-format request), and strips line-leading field labels, option letters and `None` placeholders with generic regular expressions. It reads no benchmark configuration, template or dataset file. The resulting text is embedded with bge-small-en-v1.5 [17] run locally; the embedding $e(x)$ is unit-normalised.

Correctness. For every candidate m we fit a logistic regression on $e(x)$ to predict $s_m(x) \geq 0.5$, and we take a similarity-weighted vote over the $k = 50$ nearest calibration items $\mathcal{N}(x)$:

$$\hat{p}_m(x) = \tfrac{1}{2}\,\sigma\Big(w_m^\top e(x) + b_m\Big) + \tfrac{1}{2} \sum_{i \in \mathcal{N}(x)} \omega_i \, s_m(x_i), \qquad \omega_i \propto \max\Big(0, e(x)^\top e(x_i)\Big)^4. \tag{4}$$

Cost. The expected cost is the same neighbour average of the billed cost,

$$\hat{c}_m(x) = \sum_{i \in \mathcal{N}(x)} \omega_i \, c_m(x_i), \tag{5}$$

so it reflects how long each candidate actually answers, and reasons, on similar questions.

Calibration weights. Calibration items come from several sources of very different size. Every source receives equal total weight, both as sample weights in the logistic regressions and when evaluating candidate pools and λ. To make the selection robust to difficulty, we also evaluate under weights $w_i \propto 1 + \alpha \, d_i$ renormalised within each source, where d_i is the fraction of 13 reference models that answer item i incorrectly, and average the resulting scores over $\alpha \in \{0, 1, 2, 4\}$. This uses only our calibration data.

Pool and λ. Starting from the empty pool, greedy forward selection adds the candidate that most increases the cross-validated objective (best λ on a grid, averaged over α) and stops when no candidate adds more than 0.05 points. From 12 candidates it selected five, in this order: gemini-3.8-flash at low reasoning effort, gemma-4-31b, glm-5.3-flash, gpt-6-luna and mimo-v2.6-flash, with $\lambda = 30$ USD^{-1}.

5 Experimental setup

Calibration data. We built calibration items from the same 23 public benchmarks RouterArena draws on (MMLU-Pro [16], LiveCodeBench [8], PubMedQA, MedMCQA, SuperGLUE, WMT19 and others), formatted as RouterArena formats its items. Every RouterArena item from the full, sub_10 and robustness splits was excluded by identifier and by normalised question text; the overlap is zero. Items were sampled per source with fixed quotas. Of 3,687 items collected, 3,562 have graded answers from all 13 reference models (35 source groups, 153 code problems).

Responses and grading. Each candidate was called through OpenRouter with exactly the request RouterArena sends (one user message, no temperature, token limit or reasoning parameter), plus the reasoning-effort parameter for effort variants. Answers were graded with RouterArena's own scoring code; code answers were executed in a container without network access using RouterArena's LiveCodeBench harness. Costs use the table prices of the RouterArena submission, which are OpenRouter list prices on 2026-10-07.

Candidates. Thirteen (model, effort) configurations were evaluated on the full calibration set (Table 3). Nine further models, including frontier models, were evaluated on a 325-item pilot subset (Table 4).

6 Results on RouterArena

Table 1 reports the RouterArena automated evaluation of the submitted configuration on all 8,400 queries. ARGROUTER reaches an arena score of 76.30, with accuracy 79.36% at $0.546 per 1,000 queries and a robustness score of 67.6. One of the 8,400 generations returned no answer (the model's reasoning reached the provider's maximum output length) and is scored as incorrect.

Router	Arena score	Accuracy (%)	$ / 1K queries
argrouter (ours)	**76.30**	79.36	0.546
KT-ModelRouter	76.28	78.14	0.27
Sqwish Router	76.21	79.76	0.70
Divyam	75.85	78.59	0.48
Cross-Router	75.75	78.14	0.40
vLLM Semantic Router	74.86	77.18	0.42
Azure Model Router	70.42	72.94	0.73
GPT-5 (single model)	64.32	73.96	10.02
Not Diamond	57.29	60.83	4.10
RouteLLM	48.07	47.04	0.27

Table 1: RouterArena, full split (8,400 queries). Other rows are the public leaderboard on 2026-10-07. Our row is the benchmark's automated evaluation of the submitted predictions; the submission is under review. With $n = 8{,}400$, the standard error of accuracy is about 0.45 points, so differences of a few hundredths in arena score are not a meaningful ranking.

Figure 1 places the result among all published routers. ARGROUTER lies outside their accuracy–cost frontier: every published router that is cheaper is less accurate, and the only more accurate one costs 28% more.

The router sends 44.1% of queries to gemini-3.8-flash at low effort, which accounts for 79% of the cost, and 33.3% to gemma-4-31b (Figure 2). Table 2 breaks accuracy down by source. Code problems are routed mostly to cheap models that are strong at code, and reach 90.9% accuracy.

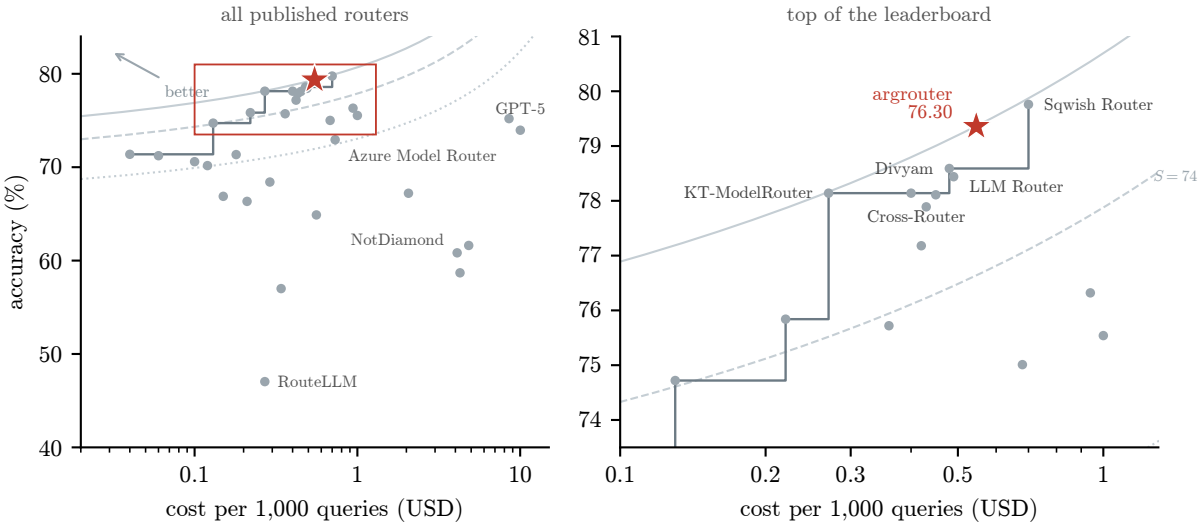


Figure 1: RouterArena: accuracy against cost. Left, all published routers; right, the boxed region. Grey points are published routers, the step line is their accuracy–cost frontier and the star is ARGROUTER. Curves are constant arena score S (Eq. 2). At its cost, no published router is as accurate as ARGROUTER.

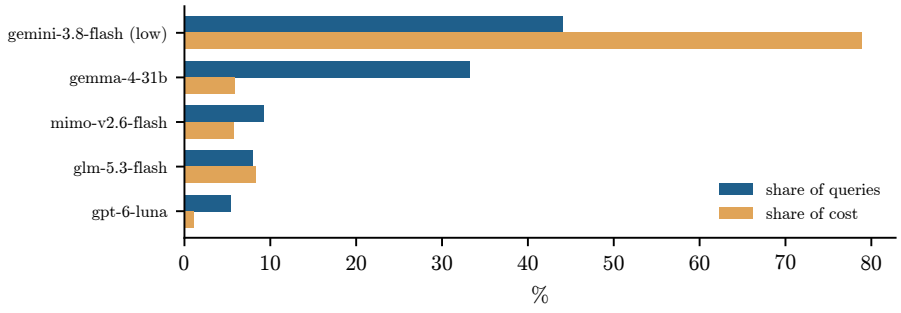


Figure 2: Share of RouterArena queries and of cost per candidate.

Source	Queries	Accuracy (%)
MMLUPro	2528	85.2
OpenTDB	887	96.5
QANTA	644	56.4
PubMedQA	466	79.0
ArcMMLU	396	83.6
LiveCodeBench	385	90.9
NarrativeQA	383	50.3
GeoBench	330	92.7
MedMCQA	304	81.2
Ethics	283	75.3
MusicTheoryBench	221	73.3
MMLU	181	92.3
MathQA	158	89.2
ChessInstruct	148	62.8
other (22 sources)	1086	67.0

Table 2: RouterArena accuracy by source, computed locally with RouterArena's scorers on the submitted predictions (local overall accuracy 79.43% versus 79.36% in the automated evaluation; the gap comes from code execution).

7 Analysis

All numbers in this section are five-fold cross-validated on the calibration set with sources weighted equally. They are not comparable in absolute value to RouterArena, whose items were selected by difficulty.

List price mis-ranks models. Table 3 and Figure 3 show single-model accuracy, cost and output length. deepseek-v4-flash has the lowest input price of all candidates yet is the most expensive to run ($3.59 per 1,000 queries) because of its long reasoning tail; glm-5.3-flash costs fourteen times as much as gemma-4-31b although its output price is less than twice as high. A router that ranks by price per token would prefer exactly the candidates that are expensive in practice.

Candidate	$ / M tokens (in / out)	Out. tok. median	p90	Acc. (%)	$ / 1K
gemini-3.8-flash (low)†	0.750 / 3.75	141	505	75.4	1.342
gemini-3.8-flash (default)	0.750 / 3.75	254	1021	75.4	2.775
gemini-3.8-flash (minimal)	0.750 / 3.75	141	516	75.3	1.307
deepseek-v4-flash	0.018 / 1.28	167	4361	72.0	3.585
gemini-3-flash (off)	0.500 / 3.00	84	245	71.7	0.570
glm-5.3-flash†	0.150 / 0.50	271	4068	71.6	2.109
mimo-v2.6-flash†	0.140 / 0.28	183	1556	69.9	0.637
gemma-4-31b†	0.080 / 0.35	76	309	69.2	0.154
gemma-4-26b-a4b	0.090 / 0.30	100	416	67.1	0.172
gemini-3.1-flash-lite (off)	0.250 / 1.50	67	188	66.5	0.221
qwen3-235b-a22b	0.071 / 0.10	77	169	66.5	0.035
gpt-oss-120b	0.037 / 0.17	196	1093	66.3	0.102
gpt-6-luna†	0.100 / 0.50	99	622	66.2	0.222

Table 3: Single candidates on the 3,562 calibration items (sources weighted equally). Output tokens include reasoning. (low), (minimal), (off) denote reasoning effort. † selected for the pool.

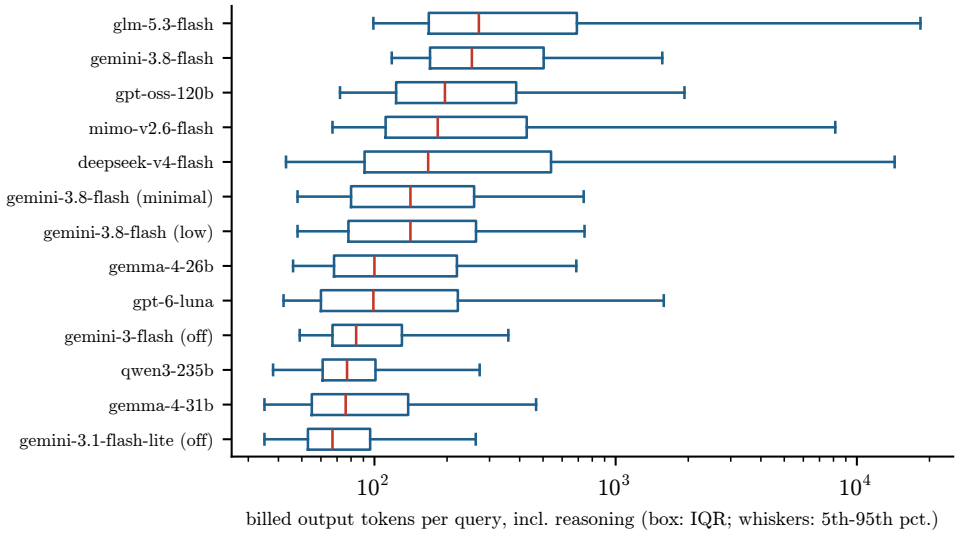


Figure 3: Billed output tokens per query on the calibration set. The spread across models, and the heavy tail of some, is what list prices do not capture.

Reasoning effort is a cheap lever. gemini-3.8-flash at its default effort and at low effort reach the same accuracy (75.4%), at $2.78 and $1.34 per 1,000 queries. Minimal effort is equivalent to low. Treating effort as part of the candidate set lets the router keep the model's accuracy at half its default cost; the selection procedure kept the low-effort variant and dropped the default.

Frontier models do not help on this distribution. On the 325-item pilot (Table 4), claude-opus-5.5 is as accurate as gemini-3.8-flash at default effort (78.8% versus 78.7%) at almost three times the cost; the other frontier models are less accurate. Accuracy on this query mix is limited by answer format, label noise and knowledge items rather than by model capability, so the candidates that matter are cheap ones.

Model	Accuracy (%)	$ / 1K queries
claude-opus-5.5	78.8	6.32
gemini-3.8-flash (default)	78.7	2.30
gemini-3.8-flash (low)	77.5	1.17
claude-sonnet-5.5	77.2	2.85
qwen3.8-max	76.4	14.69
gpt-6.1-sol	76.4	1.58
gemini-3-flash (off)	75.0	0.51
grok-4.7	74.3	21.06
gemma-4-31b	70.6	0.21

Table 4: Pilot subset of 325 calibration items (unweighted).

Routing gain and ablations. Table 5 compares the router with the best single candidates and with variants of its components. The router improves the cross-validated arena score by about 1.1 points over the best single candidate, at less than half its cost, and closes about a sixth of

the gap to the hindsight oracle over the same pool. Routing on question content alone scores slightly higher than routing on the full prompt, so removing the benchmark's instruction text costs nothing. Larger embedding models (qwen3-embedding-8b, text-embedding-3-large) are within 0.1 points of bge-small or worse, and combining the logistic regression with the neighbour vote is better than either alone. Figure 4 places the router's λ sweep among the single candidates; Table 6 in Appendix A lists it.

Variant ($\lambda = 30$)	Acc. (%)	\$ / 1K	Arena	Arena, α=1
content only, bge-small, LR+kNN (submitted)	75.1	0.57	72.59	68.81
content only, bge-small, LR only	73.8	0.54	71.57	67.62
content only, bge-small, kNN only	74.4	0.71	71.68	67.83
full prompt, bge-small, LR+kNN	74.7	0.58	72.27	68.47
full prompt, qwen3-embedding-8b, LR+kNN	73.9	0.52	71.73	67.66
full prompt, text-embedding-3-large, LR+kNN	74.6	0.52	72.33	68.43
single model: gemini-3.8-flash (low)	75.4	1.34	71.44	67.72
single model: gemma-4-31b	69.2	0.15	69.02	64.89
oracle over the pool (hindsight)	81.3	0.53	77.94	–

Table 5: Cross-validated ablations on the calibration set. "Arena, α=1" weights items towards those most reference models get wrong. Embedding-model rows use full-prompt embeddings, which were available for all three models.

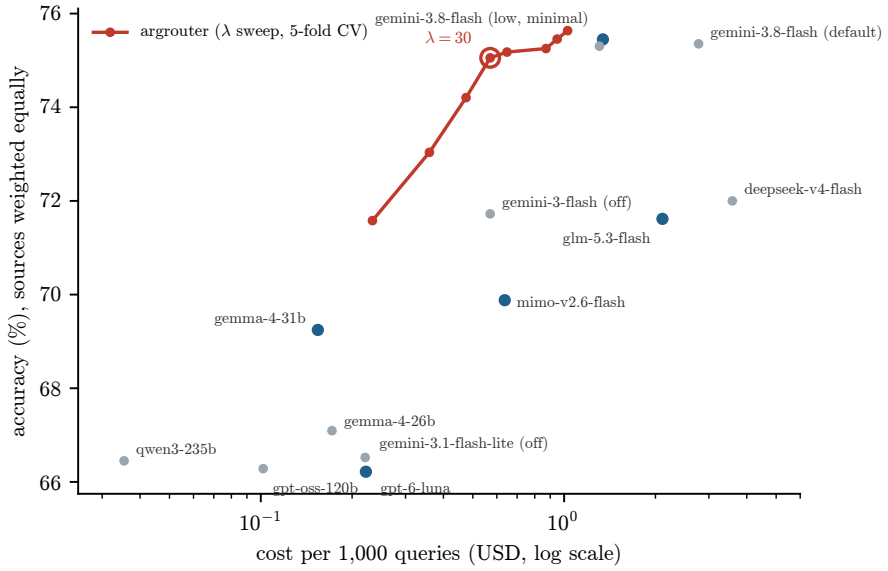


Figure 4: Accuracy against cost on the calibration set. Blue: candidates in the pool; grey: other candidates; red: the router as λ varies.

Provider choice for a fixed model. Before model routing, we measured provider routing: the same open-weight model is served by many providers at prices differing up to tenfold. Over 377 billed calls on four models, OpenRouter's default provider routing cost about two thirds more than the price-aware strategies, but ranking providers by expected cost was within noise of OpenRouter's

own price sort and of ranking by input plus output price. Provider choice therefore matters, but it is a solved problem once any price-aware sort is enabled; model and effort choice is where the remaining gain lies.

8 Benchmark hygiene

RouterArena forbids training, fitting or tuning any router component on its data, and its maintainers have ruled that routing on the harness's per-dataset instruction text, or weighting by RouterArena's per-dataset proportions, falls under that rule. The submitted configuration follows both rulings: it reads only question content and weights sources equally. During development we evaluated two earlier configurations on RouterArena that embedded the full prompt and weighted selection by the benchmark's source proportions; we discarded them for this reason. The configuration reported here was fixed on calibration data before its RouterArena run, and nothing was changed after seeing its result. The calibration items were sampled with per-source quotas set from the benchmark's source sizes; equal source weighting removes their influence on fitting and selection. Four generations initially returned no answer; following the evaluation bot's instruction, they were regenerated once with the same model and request, three produced answers and one remains a failure.

9 Limitations

The evaluation covers one benchmark whose items are mostly short-answer academic questions; a router tuned to it may not transfer to conversational or agentic traffic, where output lengths and the value of frontier models differ. Prices drift: the listed input price of one candidate rose fourfold during this work, and our table uses the prices on the day of submission. Generations are stochastic, and a reasoning model occasionally exhausts its output budget without answering. The margin to the best published router is far below the sampling error of the benchmark. Finally, the training pipeline is not public; the inference library is available at `https://github.com/Harunercul/argrouter`.

10 Conclusion

Routing on what a model actually bills, and letting the router choose how hard the model thinks, turns cheap models with short answers into the backbone of a competitive router. On RouterArena this reaches 76.30 at $0.546 per 1,000 queries using five inexpensive candidates and a router that reads only the question.

Acknowledgements

We thank the RouterArena maintainers for an open benchmark with automated evaluation. Claude (Anthropic) was used extensively for implementation, experiment orchestration and drafting; the author reviewed all content and takes full responsibility for it.

References

[1] Pranjal Aggarwal, Aman Madaan, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam

Upadhyay, Manaal Faruqui, and Mausam. AutoMix: Automatically mixing language models. *arXiv preprint arXiv:2310.12963*, 2023.

[2] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to use large language models while reducing cost and improving performance. *arXiv preprint arXiv:2305.05176*, 2023.

[3] Shuhao Chen, Weisen Jiang, Baijiong Lin, James T. Kwok, and Yu Zhang. RouterDC: Query-based router by dual contrastive learning for assembling large language models. *arXiv preprint arXiv:2409.19886*, 2024.

[4] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-efficient and quality-aware query routing. In *International Conference on Learning Representations (ICLR)*, 2024.

[5] Tao Feng, Yanzhen Shen, and Jiaxuan You. GraphRouter: A graph-based router for LLM selections. *arXiv preprint arXiv:2410.03834*, 2024.

[6] Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. RouterBench: A benchmark for multi-LLM routing system. *arXiv preprint arXiv:2403.12031*, 2024.

[7] Keke Huang, Yimin Shi, Dujian Ding, Yifei Li, Yang Fei, Laks Lakshmanan, and Xiaokui Xiao. ThriftLLM: On cost-effective selection of large language models for classification queries. *arXiv preprint arXiv:2501.04901*, 2025.

[8] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[9] Yifan Lu, Rixin Liu, Jiayi Yuan, Xingqi Cui, Shenrun Zhang, Hongyi Liu, and Jiarong Xing. RouterArena: An open platform for comprehensive comparison of LLM routers. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.00202.

[10] Not Diamond. Not Diamond: AI model router. https://www.notdiamond.ai, 2026. Accessed 2026-10-07.

[11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

[12] OpenRouter. OpenRouter: models and pricing API. https://openrouter.ai/docs, 2026. Accessed 2026-10-07.

[13] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[14] Seamus Somerstep, Felipe Maia Polo, Allysson Flavio Melo de Oliveira, Prattyush Mangal, Mírian Silva, Onkar Bhardwaj, Mikhail Yurochkin, and Subha Maity. CARROT: A cost aware rate optimal router. *arXiv preprint arXiv:2502.03261*, 2025.

[15] vLLM Semantic Router Team. vLLM semantic router. https://github.com/vllm-project/semantic-router, 2025.

[16] Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.

[17] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023.

A Additional results

λ	Acc. (%)	\$ / 1K	Arena (α=0)	Arena (α=1)	Arena (α=2)
0	75.6	1.03	72.11	68.39	65.95
5	75.5	0.95	72.10	68.33	65.86
10	75.3	0.87	72.08	68.32	65.86
20	75.2	0.65	72.50	68.76	66.31
30	75.1	0.57	72.59	68.81	66.33
50	74.2	0.48	72.13	68.20	65.61
80	73.0	0.36	71.48	67.39	64.68
120	71.6	0.23	70.70	66.58	63.84

Table 6: The router's λ sweep (cross-validated). $\lambda = 30$ is best under every difficulty weighting.